

Linux简单网络服务



一、syslogd日志服务

(1.1) 简介

什么是 syslogd?

syslog 是 Linux 系统中默认使用的系统日志服务，在CentOS 7.x中日志服务已经由rsyslogd取代了原先的syslogd服务。rsyslogd日志服务更加先进，功能更多。但是不论该服务的使用，还是日志文件的格式其实都是和syslogd服务相兼容的，所以学习起来基本和syslogd服务一致

rsyslogd的特点:

- 基于TCP网络协议传输日志信息。
- 有日志消息的及时分析框架。
- 后台数据库。
- 配置文件中可以写简单的逻辑判断。

rsyslogd日志格式:

◆基本日志格式包含以下四列:

- 事件产生的时间;
- 发生事件的服务器的主机名;
- 产生事件的服务名或程序名;
- 事件的具体信息。

rsyslogd作用:

- (1)本地系统日志管理

rsyslogd 作为 Linux 系统的默认日志守护进程，负责收集、分类、存储系统本机各类日志信息。

日志来源：操作系统本地产生的日志信息，如以下系统日志：

日志文件	说 明
/var/log/cron	记录了系统定时任务相关的日志。
/var/log/cups/	记录打印信息的日志
/var/log/dmesg	记录了系统在开机时内核自检的信息。也可以使用dmesg命令直接查看内核自检信息。
/var/log/btmp	记录错误登录的日志。这个文件是二进制文件，不能直接vi查看，而要使用lastb命令查看，命令如下： <pre>[root@localhost log]# lastb root tty1 Tue Jun 4 22:38 - 22:38 (00:00) #有人在6月4日22:38使用root用户，在本地终端1登录错误</pre>
/var/log/lastlog	记录系统中所有用户最后一次的登录时间的日志。这个文件也是二进制文件，不能直接vi，而要使用lastlog命令查看。
/var/log/maillog	记录邮件信息。
/var/log/message	记录系统重要信息的日志。这个日志文件中会记录Linux系统的绝大多数重要信息，如果系统出现问题时，首先要检查的就应该是这个日志文件。
/var/log/secure	记录验证和授权方面的信息，只要涉及账户和密码的程序都会记录。比如说系统的登录，ssh的登录，su切换用户，sudo授权，甚至添加用户和修改用户密码都会记录在这个日志文件中。
/var/log/wtmp	永久记录所有用户的登录、注销信息，同时记录系统的启动、重启、关机事件。同样这个文件也是一个二进制文件，不能直接vi，而需要使用last命令来查看。
/var/run/utmp	记录当前已经登录的用户的信息。这个文件会随着用户的登录和注销而不断变化，只记录当前登录用户的信息。同样这个文件不能直接vi，而要使用w，who，users等命令来查询。

应用场景：

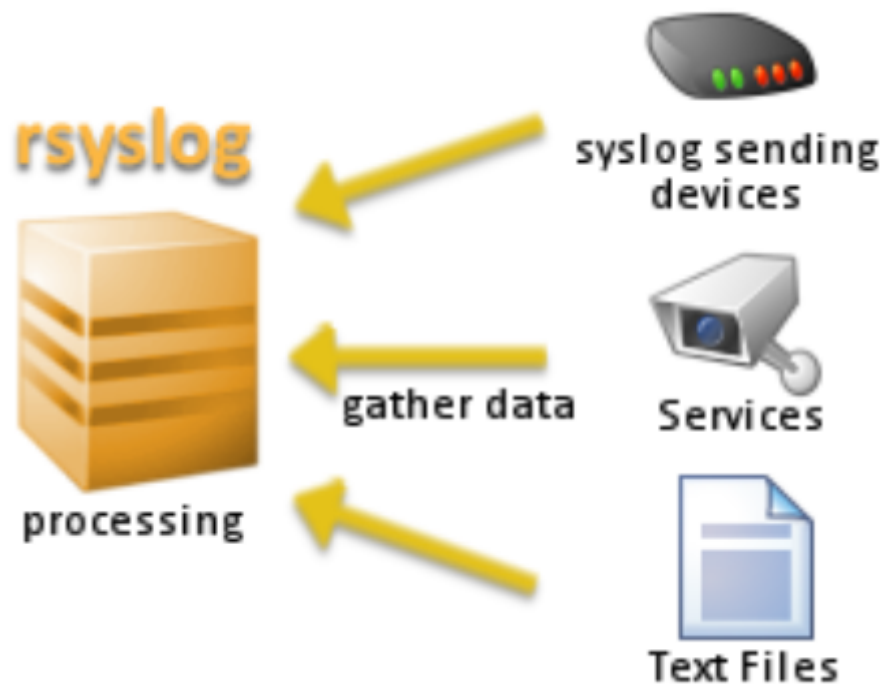
- 排查系统服务崩溃
通过/var/log/messages日志和 /var/log/dmesg 进行排查
- 监控登录失败记录
通过/var/log/secure 日志进行排查

• (2)远程日志收集

rsyslogd 能监听网络端口，通过TCP/UDP协议接收来自其他主机、网络设备、应用系统发送过来的日志，并分类保存。

日志来源：

- 其他 Linux 服务器（通过 rsyslog 发送）
- Windows 系统（使用 NxLog、Snare 等工具）
- 网络设备（如 Cisco、H3C、FortiGate）
- IoT、安全设备（如 IDS/IPS、摄像头）



应用场景：

- 日志备份
将设备产生的日志进行异地备份
- 日志投递
部分存储功能不足的设备将日志存储到异地服务器

• (3)推送服务日志（转发日志到其他系统）

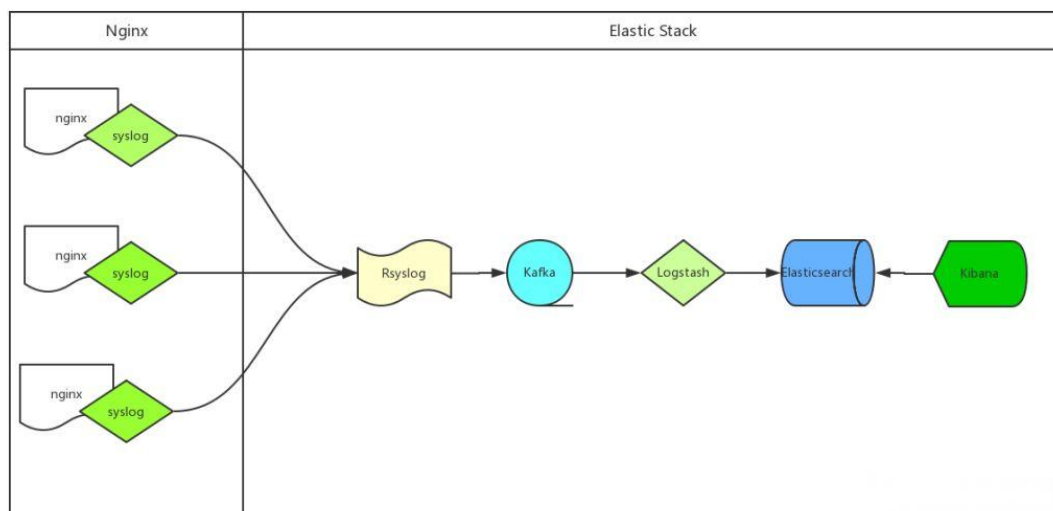
rsyslogd 不仅能收集日志，还能将日志 主动推送 到其他主机、日志服务器或平台，实现跨系统日志聚合。

日志推送目标：

- 日志分析平台：ELK Stack（Logstash/Elasticsearch）

- 消息队列：Kafka、RabbitMQ
- 安全审计系统：SIEM、SOC 平台
- 数据库：MySQL、PostgreSQL（做合规归档）

常用的是推送到ELK上，进行日志可视化



企业常用：日志集中化管理，使用rsyslogd进行日志推送到ELK上，最终进行可视化查询

(1.2) rsyslog服务管理

1.2.1 服务状态确认

rsyslog是默认安装的服务，我们可用通过 `systemctl` 或 `ps` 确认服务状态

```
systemctl status rsyslog
# 或
ps aux|grep rsyslog
```

```
[root@theon opt]# systemctl status rsyslog
● rsyslog.service - System Logging Service
   Loaded: loaded (/usr/lib/systemd/system/rsyslog.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2025-06-16 18:56:05 CST; 3h 36min ago
     Docs: man:rsyslogd(8)
           http://www.rsyslog.com/doc/
   Main PID: 988 (rsyslogd)
      Tasks: 3
     Memory: 4.1M
    CGroup: /system.slice/rsyslog.service
            └─988 /usr/sbin/rsyslogd -n

Jun 16 18:55:50 theon systemd[1]: Starting System Logging Service...
Jun 16 18:56:05 theon rsyslogd[988]: [origin software="rsyslogd" swVersion="8.24.0" x-pid="988" x-info="http://www.rsyslog.com"] start
Jun 16 18:56:05 theon systemd[1]: Started System Logging Service.
[root@theon opt]# ps aux|grep rsyslog
root      988  0.0  0.4 220588  8668 ?        Ssl  18:55   0:00 /usr/sbin/rsyslogd -n
root     3641  0.0  0.0 112812   980 pts/1    R+   22:33   0:00 grep --color=auto rsyslog
[root@theon opt]#
```

1.2.2 rsyslog日志配置

rsyslog服务的配置文件位于 `/etc/rsyslog.conf`，里面规定了rsyslog服务的日志配置规则

```
# Log all kernel messages to the console.
# Logging much else clutters up the screen.
#kern.*                                /dev/console

# Log anything (except mail) of level info or higher.
# Don't log private authentication messages!
*.info;mail.none;authpriv.none;cron.none    /var/log/messages

# The authpriv file has restricted access.
authpriv.*                                /var/log/secure

# Log all the mail messages in one place.
mail.*                                     -/var/log/maillog

# Log cron stuff
cron.*                                    /var/log/cron

# Everybody gets emergency messages
*.emerg                                   :omusrmsg:*

# Save news errors of level crit and higher in a special file.
uucp,news.crit                            /var/log/spooler

# Save boot messages also to boot.log
local7.*                                  /var/log/boot.log
```

我们以常见的登录日志为例：

```
authpriv.*                                /var/log/secure
```

主要由四个部分组成：

- 1.服务名称： `authpriv`

服务名称是操作系统已经设定好的模块，除了 `local*` 类，你填写了什么样的服务，操作系统就会产生什么样的日志信息

服务名称和对应系统产生的信息，如下表所示：

服务名称	说 明
auth	安全和认证相关消息（不推荐使用authpriv替代）
authpriv	安全和认证相关消息（私有的）
cron	系统定时任务cron和at产生的日志
daemon	和各个守护进程相关的日志
ftp	ftp守护进程产生的日志
kern	内核产生的日志（不是用户进程产生的）
local0-local7	为本地使用预留的服务
lpr	打印产生的日志

mail	邮件收发信息
news	与新闻服务器相关的日志
syslog ⁺	有syslogd服务产生的日志信息（虽然服务名称已经改为rsyslogd，但是很多配置都还是沿用了syslogd的，这里并没有修改服务名）。
user	用户等级类别的日志信息
uucp	uucp子系统的日志信息，uucp是早期linux系统进行数据传递的协议，后来也常用在新闻组服务中。

- 2.连接符号: .

连接符号代表日志生成的条件

- “.” 代表只要比后面的等级高的（包含该等级）日志都记录下来。比如：“cron.info”代表cron服务产生的日志，只要日志等级大于等于info级别，就记录
- “.=” 代表只记录所需等级的日志，其他等级的都不记录。比如：“*=emerg”代表人和日志服务产生的日志，只要等级是emerg等级就记录。这种用法及少见，了解就好
- “!” 代表不等于，也就是除了该等级的日志外，其他等级的日志都记录。

- 3.日志记录优先级: *

等级名称	说明
debug	一般的调试信息说明
info	基本的通知信息
notice	普通信息，但是有一定的重要性
warning	警告信息，但是还不回影响到服务或系统的运行
err	错误信息，一般达到err等级的信息以及可以影响到服务或系统的运行了。
crit	临界状况信息，比err等级还要严重
alert	警告状态信息，比crit还要严重。必须立即采取行动
emerg ⁺	疼痛等级信息，系统已经无法使用了

- 4.日志存放位置: /var/log/secure

日志信息存放在那个文件，也可以是其他设备、网络地址信息

日志记录位置

- ◆ 日志文件的绝对路径，如 “/var/log/secure”
- ◆ 系统设备文件，如 “/dev/lp0” **不常用**
- ◆ 转发给远程主机，如 “@192.168.0.210:514”
- ◆ 用户名，如 “root”
- ◆ 忽略或丢弃日志，如 “~”

常用方式

搭建日志服务器

1.2.3 rsyslog日志配置范例

- 利用自定义服务类型local0-7，建立服务日志

比如我们建立sshd服务的日志

- ①.修改sshd_config配置文件，将ssh日志类型改为local0

```
vim /etc/ssh/sshd_config  
SyslogFacility local0 #AUTHPRIV 改为local0
```

```
#  
#Port 22  
#AddressFamily any  
#ListenAddress 0.0.0.0  
#ListenAddress ::  
  
HostKey /etc/ssh/ssh_host_rsa_key  
#HostKey /etc/ssh/ssh_host_dsa_key  
HostKey /etc/ssh/ssh_host_ecdsa_key  
HostKey /etc/ssh/ssh_host_ed25519_key  
  
# Ciphers and keying  
#RekeyLimit default none  
  
# Logging  
#SyslogFacility AUTH  
SyslogFacility local0  
#LogLevel INFO  
  
# Authentication:  
  
#LoginGraceTime 2m  
#PermitRootLogin yes
```

- ②.修改日志服务配置文件，加入sshd日志路径

```
vim /etc/rsyslog.conf
#添加:
local0.info                                /var/log/sshd.log
```

```
# Save news errors of level crit and higher in a special file.
uucp,news.crit                                /var/log/spooler

# Save boot messages also to boot.log
local7.*                                      /var/log/boot.log

# save ssh log
local0.info                                /var/log/sshd.log

# ### begin forwarding rule ###
# The statement between the begin ... end define a SINGLE forwarding
# rule. They belong together, do NOT split them. If you create multiple
# forwarding rules, duplicate the whole block!
# Remote Logging (we use TCP for reliable delivery)
#
# An on-disk queue is created for this action. If the remote host is
```

③.重启日志服务和sshd服务

```
systemctl restart sshd
systemctl restart rsyslog
# 我们在重启sshd产生日志
systemctl restart sshd
# 查看rsyslog记录的sshd日志
cat /var/log/sshd.log
```

```
[root@theon opt]# cat /var/log/sshd.log
Jun 16 22:53:44 theon sshd[3697]: Received signal 15; terminating.
Jun 16 22:53:44 theon sshd[3716]: Server listening on 0.0.0.0 port 22.
Jun 16 22:53:44 theon sshd[3716]: Server listening on :: port 22.
[root@theon opt]#
```

• 基于日志记录位置，建立简单的日志服务器

①.修改服务端的日志服务的配置文件

```
vim /etc/rsyslog.conf
#在服务端打开tcp数据发送模块，然后保存退出
$ModLoad imtcp
$InputTCPServerRun 514
```



```
##$ModLoad imklog # reads kernel messages (the same are read from journald)
##$ModLoad immark # provides --MARK-- message capability

# Provides UDP syslog reception
##$ModLoad imudp
##$UDPServerRun 514

# Provides TCP syslog reception
$ModLoad imtcp
$InputTCPServerRun 514

##### GLOBAL DIRECTIVES #####

# Where to place auxiliary files
$WorkDirectory /var/lib/rsyslog

# Use default timestamp format
$ActionFileDefaultTemplate RSYSLOG_TraditionalFileFormat
```

UDP日志接收

TCP日志接收

②.重启日志服务

```
systemctl restart rsyslog
# 查看rsyslog监听端口
netstat -tulnp
```

```
[root@theon opt]# netstat -tulnp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:514             0.0.0.0:*               LISTEN      3746/rsyslogd
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      3716/sshd
tcp        0      0 127.0.0.1:25            0.0.0.0:*               LISTEN      1277/master
tcp6       0      0 :::514                  :::*                    LISTEN      3746/rsyslogd
tcp6       0      0 :::22                   :::*                    LISTEN      3716/sshd
tcp6       0      0 :::1:25                 :::*                    LISTEN      1277/master
udp        0      0 0.0.0.0:68              0.0.0.0:*               LISTEN      3252/dhclient
[root@theon opt]#
```

扩展：其他设备/服务端如何向此日志服务器传递日志信息？

- 1、window通过NxLog、Snare传递
- 2、设备web管理页面的 日志投递 功能
- 3、其他linux操作系统利用rsyslog投递（比如在配置文件日志配置参数的地址中填写：
@@192.168.0.117:514）

• rsyslog推送日志到kafka说明

①.安装模块

```
yum -y install rsyslog-kafka
#安装完成后确认相关推送模块是否存在
ls /usr/lib*/rsyslog/ | grep imfile
ls /usr/lib*/rsyslog/ | grep omkafka
```

②.修改服务端的日志服务的配置文件

```
vim /etc/rsyslog.conf
```

```

#添加加载模块、配置日志格式、推送动作参数
module(load="omkafka")
module(load="imfile")
template(name="json-log" type="list") {
    constant(value="{")
    constant(value="\">@timestamp\":"") property(name="timereported"
dateFormat="rfc3339")
    constant(value="\","host\":"") property(name="hostname")
    constant(value="\","program\":"") property(name="programname")
    constant(value="\","msg\":"") property(name="msg" format="json")
    constant(value="}")
}
action(
    type="omkafka"
    topic="syslog_topic"
    broker="192.168.1.100:9092" # Kafka broker 地址
    template="json-log"
)

#修改后重启rsyslog
systemctl restart rsyslog

```

1.2.4 日志轮替

日志轮替：将日志切割成每天记录，并定期删除旧的日志生成新的日志。

根据合规要，旧日志，至少保留180天

日志轮替配置文件位于：`/etc/logrotate.conf` 配置文件按照相应的日志命名规则来轮替文件

1、日志文件的命名规则

- ◆ 如果配置文件中拥有“dateext”参数，那么日志会用日期来作为日志文件的后缀，例如“secure-20130605”。这样的话日志文件名不会重叠，所以也就不需要日志文件的改名，只需要保存指定的日志个数，删除多余的日志文件即可。

- ◆ 如果配置文件中没有“dateext”参数，那么日志文件就需要进行改名了。当第一次进行日志轮替时，当前的“secure”日志会自动改名为“secure.1”，然后新建“secure”日志，用来保存新的日志。当第二次进行日志轮替时，“secure.1”会自动改名为“secure.2”，当前的“secure”日志会自动改名为“secure.1”，然后也会新建“secure”日志，用来保存新的日志，以此类推。

默认使用的是dateext命名规则

```
# see "man logrotate" for details
# rotate log files weekly
weekly

# keep 4 weeks worth of backlogs
rotate 4

# create new (empty) log files after rotating old ones
create

# use date as a suffix of the rotated file
dateext

# uncomment this if you want your log files compressed
#compress

# RPM packages drop log rotation information into this directory
include /etc/logrotate.d
```

/etc/logrotate.conf 配置文件参数说明

参 数	参 数 说 明
daily	日志的轮替周期是每天
weekly	日志的轮替周期是每周
monthly	日志的轮替周期是每月
rotate 数字	保留的日志文件的个数。0指没有备份
compress	日志轮替时，旧的日志进行压缩
create mode owner group	建立新日志，同时指定新日志的权限与所有者和所属组。如create 0600 root utmp
mail address	当日志轮替时，输出内容通过邮件发送到指定的邮件地址。如mail shenc@lamp.net
missingok	如果日志不存在，则忽略该日志的警告信息
notifempty	如果日志为空文件，则不进行日志轮替
minsize 大小	日志轮替的最小值。也就是日志一定要达到这个最小值才会轮替，否则就算时间达到也不轮替
size 大小	日志只有大于指定大小才进行日志轮替，而不是按照时间轮替。如size 100k
dateext	使用日期作为日志轮替文件的后缀。如secure-20130605

范例：把nginx日志加入轮替

```
# 修改`=/etc/logrotate.conf配置文件
# 添加如下内容
/var/log/nginx/www.zwxa.com_access.log {
daily
create 0600 nginx nginx
minsize 100M
rotate 30
}
```

```
#compress

# RPM packages drop log rotation information into this directory
include /etc/logrotate.d

# no packages own wtmp and btmp -- we'll rotate them here
/var/log/wtmp {
    monthly
    create 0664 root utmp
    minsize 1M
    rotate 1
}

/var/log/btmp {
    missingok
    monthly
    create 0600 root utmp
    rotate 1
}

/var/log/nginx/www.zwxa.com_access.log {
    daily
    create 0600 nginx nginx
    minsize 100M
    rotate 30
}
```

实际上，yum安装的nginx在/etc/logrotate.d/nginx已经有默认的轮替规则了，如果你是yum安装nginx，则不用再额外写轮替规则，否则轮替会报错

轮替命令logrotate

格式：

logrotate [选项] 配置文件名

选项：

如果此命令没有选项，则会按照配置文件中的条件进行日志轮替

-v: 显示日志轮替过程。加了-v选项，会显示日志的轮替的过程

-f: 强制进行日志轮替。不管日志轮替的条件是否已经符合，强制配置文件中所有的日志进行轮替

二、SSH服务

(2.1) 服务简介

ssh（安全外壳协议）

- ◆ SSH 为 Secure Shell 的缩写，SSH 为建立在应用层和传输层基础上的安全协议。

SSH服务：

SSH端口

- ◆ SSH端口：22
- ◆ Linux中守护进程：sshd
- ◆ 安装服务：OpenSSH
- ◆ 服务端主程序：/usr/sbin/sshd
- ◆ 客户端主程序：/usr/bin/ssh

SSH加密原理

非对称加密算法（安全）

如何查看加密算法？

```
#ssh -vv user@hostname  
ssh -vv root@192.168.239.149
```

```
openssh.com,umac-128@openssh.com,hmac-sha2-256,hmac-sha2-512,hmac-sha1  
debug2: compression ctos: none,zlib@openssh.com  
debug2: compression stoc: none,zlib@openssh.com  
debug2: languages ctos:  
debug2: languages stoc:  
debug2: first_kex_follows 0  
debug2: reserved 0  
debug1: kex: algorithm: curve25519-sha256  
debug1: kex: host key algorithm: ecdsa-sha2-nistp256  
debug1: kex: server->client cipher: chacha20-poly1305@openssh.com MAC: <implicit> compression: none  
debug1: kex: client->server cipher: chacha20-poly1305@openssh.com MAC: <implicit> compression: none  
debug1: kex: curve25519-sha256 need=64 dh_need=64  
debug1: kex: curve25519-sha256 need=64 dh_need=64  
debug1: expecting SSH2_MSG_KEX_ECDH_REPLY  
debug1: Server host key: ecdsa-sha2-nistp256 SHA256:zBYKpRRMoyqotxXSkc0TJHMcN0tYoPmnBSmWDXVsS9c  
The authenticity of host '192.168.239.149 (192.168.239.149)' can't be established.  
ECDSA key fingerprint is SHA256:zBYKpRRMoyqotxXSkc0TJHMcN0tYoPmnBSmWDXVsS9c.  
ECDSA key fingerprint is MD5:bf:32:ca:9d:d0:ae:e3:fb:f2:0c:fb:b3:82:b9:a5:e9.  
Are you sure you want to continue connecting (yes/no)? ^C  
[root@theop-ent149 ~]#
```


查看ssh服务端支持哪些算法

```
sshd -T | grep -E 'cipher|mac|kex|hostkeyalgorithms'
```

```
gssapikexalgorithms gss-gex-sha1,gss-group1-sha1,gss-group14-sha1-
ciphers chacha20-poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256-ctr,aes128-gcm@openssh.com,aes256-gcm@openssh.com,aes128-cbc,aes192-cbc,aes256-cbc,blowfish-cbc
,cast128-cbc,3des-cbc
macs umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,hmac-sha1-etm@openssh.com,umac-64@openssh.com,um
ac-128@openssh.com,hmac-sha2-256,hmac-sha2-512,hmac-sha1
kexalgorithms curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nistp256,ecdh-sha2-nistp384,ecdh-sha2-nistp521,diffie-hellman-group-exchange-sha256,diffie-
hellman-group16-sha512,diffie-hellman-group18-sha512,diffie-hellman-group-exchange-sha1,diffie-hellman-group14-sha256,diffie-hellman-group14-sha1,diffie-hellman-g
roup1-sha1
hostkeyalgorithms ecdsa-sha2-nistp256-cert-v01@openssh.com,ecdsa-sha2-nistp384-cert-v01@openssh.com,ecdsa-sha2-nistp521-cert-v01@openssh.com,ssh-ed25519-cert-v01@
openssh.com,ssh-rsa-cert-v01@openssh.com,ssh-dss-cert-v01@openssh.com,ecdsa-sha2-nistp256,ecdsa-sha2-nistp384,ecdsa-sha2-nistp521,ssh-ed25519,rsa-sha2-512,rsa-sha
2-256,ssh-rsa,ssh-dss
```

(2.2) SSH配置文件

2.2.1 服务器端配置文件

/etc/ssh/sshd_config

/etc/ssh/sshd_config

◆ Port 22	端口
◆ ListenAddress 0.0.0.0 +	监听的IP
◆ Protocol 2	SSH版本选择
◆ HostKey /etc/ssh/ssh_host_rsa_key	私钥保存位置
◆ ServerKeyBits 1024	私钥的位数
◆ SyslogFacility AUTH	日志记录SSH登陆情况
◆ LogLevel INFO	日志等级
◆ GSSAPIAuthentication yes	GSSAPI认证开启

补充:

- 为了安全起见,防止SSH服务端被攻击(暴力破解攻击),可以考虑修改端口号
- GSSAPI认证:验证域名与ip,一般建议关闭,否则花费连接时的时间大大增加

安全设定建议:

◆ PermitRootLogin yes ^{no}	允许root的ssh登陆
◆ PubkeyAuthentication yes	是否使用公钥验证
◆ AuthorizedKeysFile .ssh/authorized_keys	公钥的保存位置
◆ PasswordAuthentication <u>yes</u> ^{no}	允许使用密码验证登陆
◆ PermitEmptyPasswords <u>no</u>	不允许空密码登陆

2.2.2 SSH常用命令

- 1.SSH登陆命令

Linux登陆到另外一条Linux

格式:

`ssh 用户名@需要连接的ip地址`

范例

```
ssh root@192.168.239.149
```

- 远程复制

从Linux服务器拷贝文件

格式:

下载: `scp -P 22 登陆用户@IP地址: /源路径 目的路径`

上传: `scp -P 22 登陆用户@IP地址: /目的路径 源路径`

范例

#将192.168.239.149下/root/ALL.sql文件，下载到本地/tmp目录下

```
scp -r -P 22 root@192.168.239.149:/root/ALL.sql /tmp
```

#将本地/opt/myding/docker-compose.yml文件上传到192.168.239.149的/var/tmp目录下

```
scp -r -P 22 /opt/myding/docker-compose.yml root@192.168.239.149:/var/tmp
```

- 文件传输

linux之间进行文件传输

范例:

```
sftp root@192.168.239.149
```

命令会进入交互界面，在交互界面可用执行如下命令

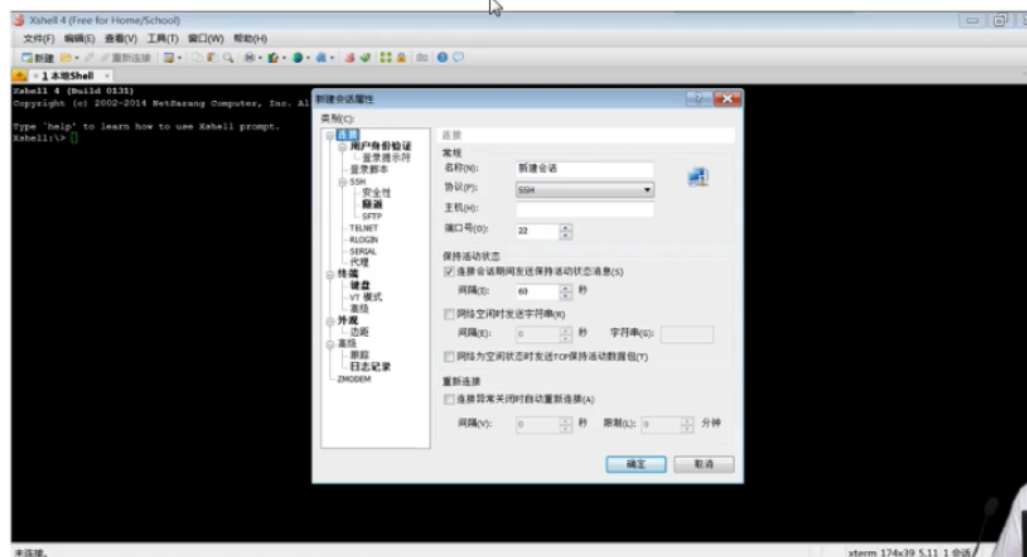
- ls	查看服务器端数据
- cd	切换服务器端目录
- ll	查看本地数据
- lcd	切换本地目录
- get	下载
- put	上传

```
[root@theon ~]# sftp root@192.168.239.149
root@192.168.239.149's password:
Connected to 192.168.239.149.
sftp> lcd /var
sftp> ls
ALL.sql                all.txt                anaconda-ks.cfg        book.sql
ddos_ex.pacp           dwwa_deploy.zip        first.sh                htop-2.2.0
sale.sql               sshd_config            stderr.txt              stdout.txt
t.txt                  w.sh                   work_20250612.sh       workspace
sftp> ll
adm backup cache crash db empty games gopher kerberos lib local lock log mail nis opt p
sftp> get all.txt
Fetching /root/all.txt to all.txt
sftp>
```

- **SSH连接工具**

- 1、xshell

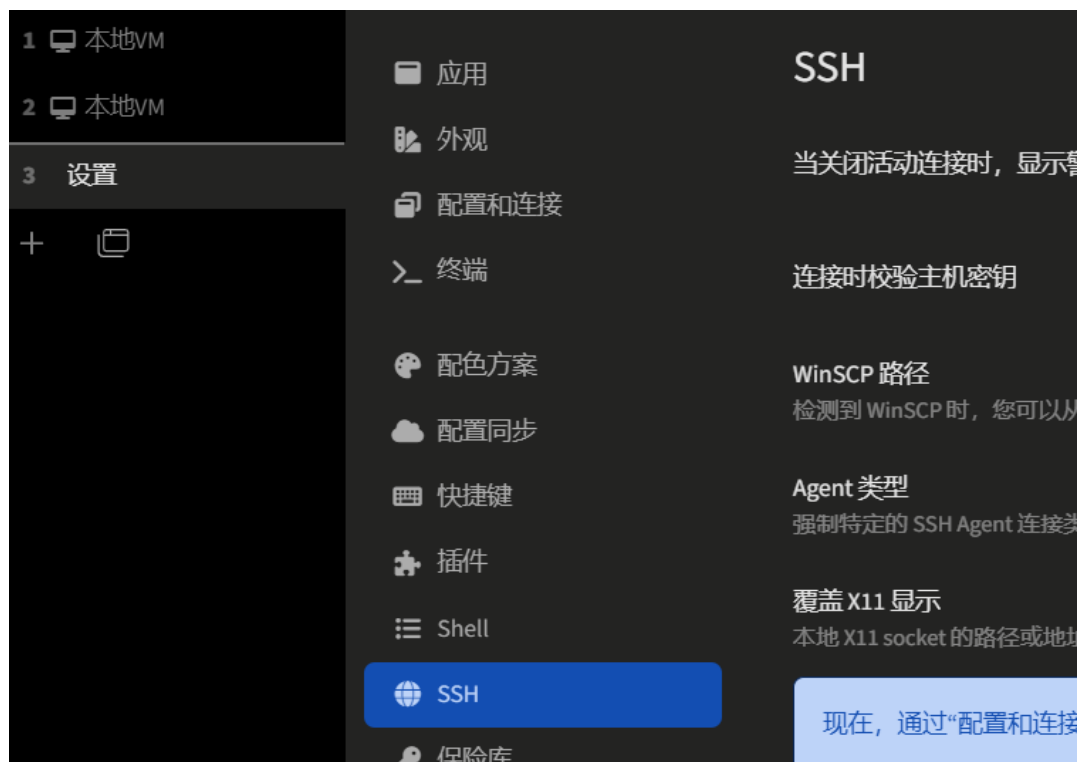
Xshell



- 2.MobaXterm



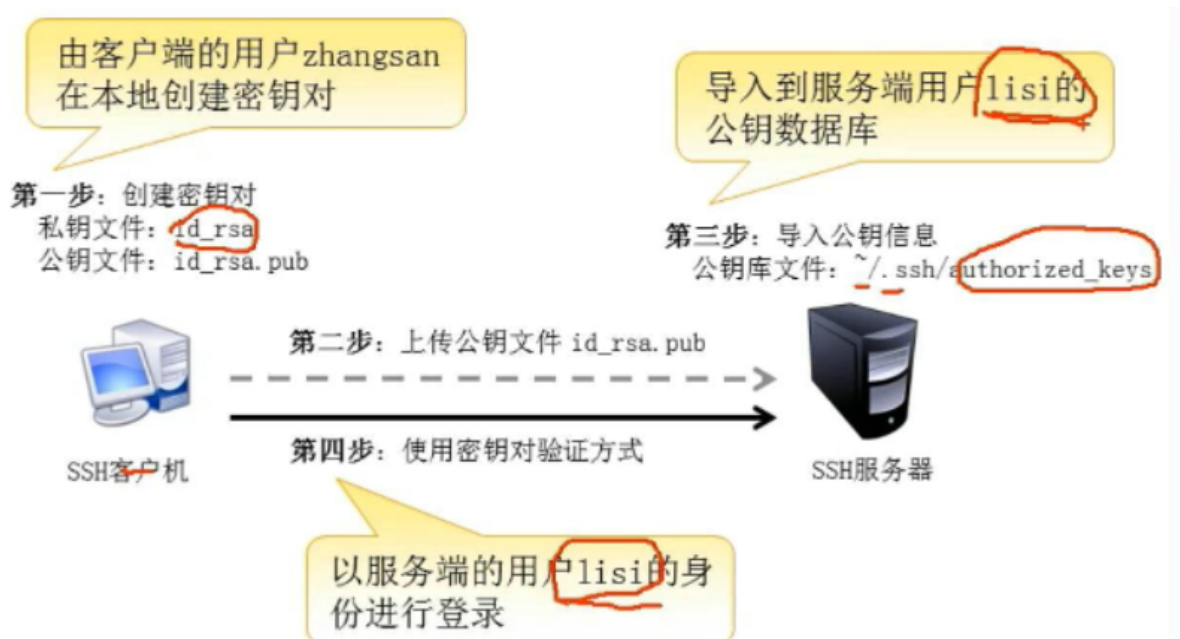
- o 3.tabby



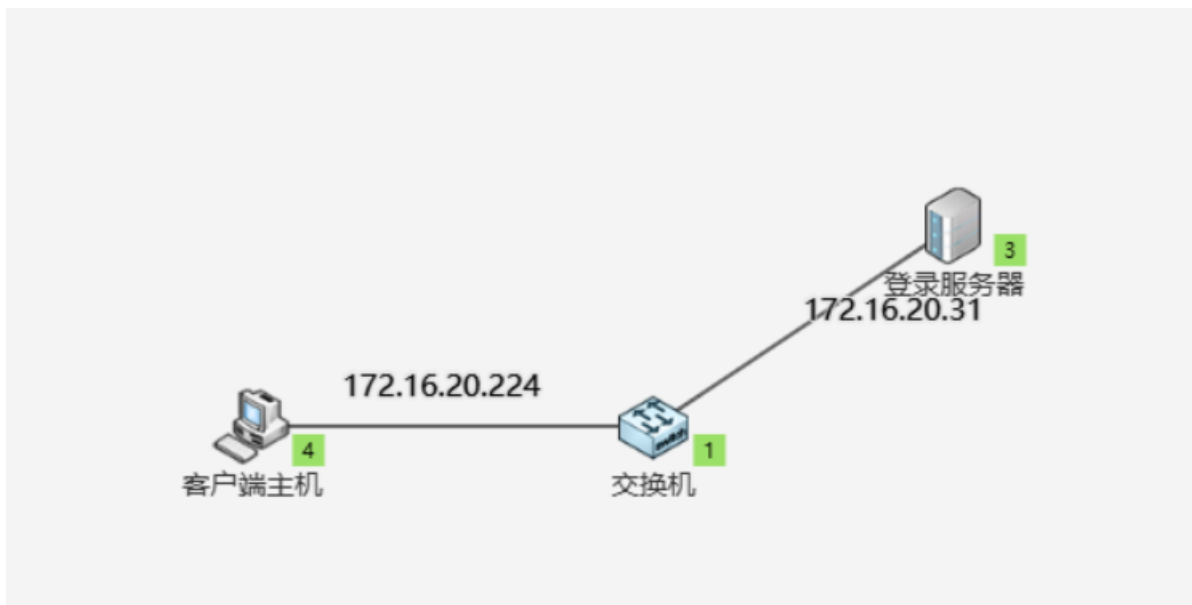
(2.3) SSH密钥对登录

ssh安全性最高、灵活的登录认证方式

密钥对验证登录部署流程图



架构图



服务器	IP
客户端主机	172.16.20.224
登录服务器	172.16.20.31

2.3.1 秘钥部署

◆ **client端:**

- ssh-keygen -t rsa

◆ **server端**

- 把公钥上传到服务器端
- cat id_rsa.pub >> /root/.ssh/authorized_keys
- chmod 600 /root/.ssh/authorized_keys

1.客户端生成密匙:

```
ssh-keygen -t rsa
```

(ssh-keygen中间无空格)
#保存位置默认 直接回车
#密码不需要 直接回车

2.客户端把公钥传到服务器端上:

```
cd ~/.ssh
scp id_rsa.pub lisi@172.16.20.31:/home/lisi
```

3.服务端将公钥保存在需要连接的用户目录下 (例如lisi) :

```
mkdir ~/.ssh
chmod 700 ~/.ssh
cat id_rsa.pub >> .ssh/authorized_keys
#(文件名不要错)
```

4.服务端改公钥权限

```
chmod 600 .ssh/authorized_keys
```

方法二：以上步骤可以在客户端简写为：

```
#秘钥对先创建好
ssh lisi@172.16.20.31 -p 22 "mkdir ~/.ssh"
ssh lisi@172.16.20.31 -p 22 "chmod 700 ~/.ssh"
cat ~/.ssh/id_rsa.pub | ssh lisi@172.16.20.31 -p 22 "cat - >>
~/.ssh/authorized_keys"
ssh lisi@172.16.20.31 -p 22 "chmod 600 ~/.ssh/authorized_keys"
```

2.3.2 sshd配置

◆修改服务器端ssh配置文件

- RSAAuthentication yes 开启RSA验证
- PubkeyAuthentication yes 是否使用公钥验证
- AuthorizedKeysFile .ssh/authorized_keys 公钥的保存位置
- PasswordAuthentication no 禁止使用密码验证登陆

下面服务端的操作都需要使用 root 登录

1.修改服务器端/etc/ssh/sshd_config配置文件

```
vi /etc/ssh/sshd_config
#去掉#PubkeyAuthentication yes前面的#号
#PasswordAuthentication yes改为PasswordAuthentication no
```

2.服务端重启ssh

```
systemctl restart sshd
```

2.3.3 登录测试

在客户端机上面执行ssh登录命令，查看是否还需要输入密码

```
ssh lisi@172.16.20.31 -p 22
```



```
Connection to 172.16.20.31 closed.  
[zhangsan@localhost .ssh]$ ssh lisi@172.16.20.31  
Last login: Thu Jun 19 08:44:00 2025 from 172.16.20.224  
[lisi@localhost ~]$
```

(2.4) SSH防止暴力破解

ssh暴力破解频发，严重关系到服务器的安全，运维人员需要重视，做好安全防护措施*

2.4.1 配置安全的sshd服务

1 密码足够的复杂，密码的长度要大于8位最好大于20位。密码的复杂度是密码要尽可能有数字、大小写字母和特殊符号混合组成，

2修改默认端口号

3 不允许root账号直接登陆，添加普通账号，授予root的sudo权限

4 不允许密码登陆，只能通过认证的秘钥来登陆系统

2.4.2 编写shell脚本防御爆破

脚本ssh_ban.sh如下

```
#!/bin/bash  
#Info: ssh防止暴力破解脚本  
#Time: 2025-06-17  
#Author: Theon  
#Version: 0.0.1  
  
# 变量  
DEFINE=5  
NUM=0  
TEMP_FILE='/tmp/blackip.txt'  
  
# main  
while true  
do  
grep 'sshd.*authentication failure' /var/log/secure | grep 'rhost=' | awk -  
F'rhost=' '{print $2}' | awk '{print $1}' | sort | uniq -c | sort -nr >  
${TEMP_FILE}  
while read count ip  
do  
IP=${ip}  
NUM=${count}  
if [ $NUM -gt $DEFINE ]  
then
```

```

        grep $IP /etc/hosts.deny >/dev/null
        if [ $? -gt 0 ]
        then
            echo "sshd:$IP" >> /etc/hosts.deny
        fi
    fi
done < ${TEMP_FILE}
sleep 30
done

```

运行脚本

```

# chmod +x /opt/ssh_ban/ssh_ban.sh
/opt/ssh_ban/ssh_ban.sh &

```

测试脚本效果

```

2025-06-17 00:46.37 /home/mobaxterm ssh root@192.168.239.149
root@192.168.239.149's password:
root@192.168.239.149's password:
root@192.168.239.149's password:
root@192.168.239.149: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).

2025-06-17 01:32.57 /home/mobaxterm ssh root@192.168.239.149
root@192.168.239.149's password:
root@192.168.239.149's password:
root@192.168.239.149's password:
root@192.168.239.149: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).

2025-06-17 01:33.16 /home/mobaxterm ssh root@192.168.239.149
root@192.168.239.149's password:
root@192.168.239.149's password:
root@192.168.239.149's password:
root@192.168.239.149: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).

2025-06-17 01:33.26 /home/mobaxterm ssh root@192.168.239.149
kex_exchange_identification: read: Software caused connection abort
banner exchange: Connection to 192.168.239.149 port 22: Software caused connection abort

[root@theon ssh_ban]# cat /etc/hosts.deny
#
# hosts.deny      This file contains access rules which are used to
#                  deny connections to network services that either use
#                  the tcp_wrappers library or that have been
#                  started through a tcp_wrappers-enabled xinetd.
#
#                  The rules in this file can also be set up in
#                  /etc/hosts.allow with a 'deny' option instead.
#
#                  See 'man 5 hosts_options' and 'man 5 hosts_access'
#                  for information on rule syntax.
#                  See 'man tcpd' for information on tcp_wrappers
#
sshd:192.168.239.1

```

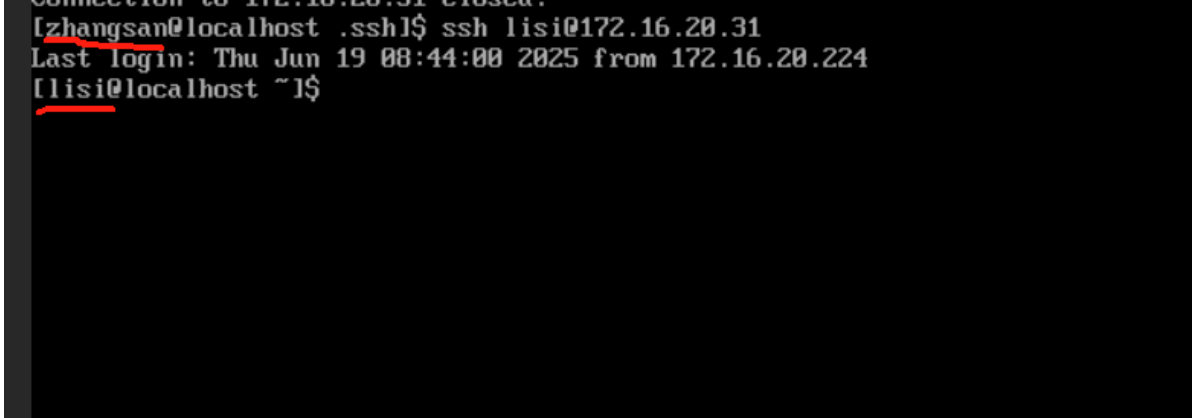
清除IP封禁效果:

```
cp -f /var/log/secure /var/log/secure.bak
cp -f /etc/hosts.deny /etc/hosts.deny.bak
> /var/log/secure
> /etc/hosts.deny
```

作业：SSH密钥登录配置

任务：请配置ssh密钥登录，同时关闭ssh的密码登录

提交内容：ssh密钥登录截图，如下范例



```
[zhangsan@localhost ~]$ ssh lisi@172.16.20.31
Last login: Thu Jun 19 08:44:00 2025 from 172.16.20.224
[lisi@localhost ~]$
```